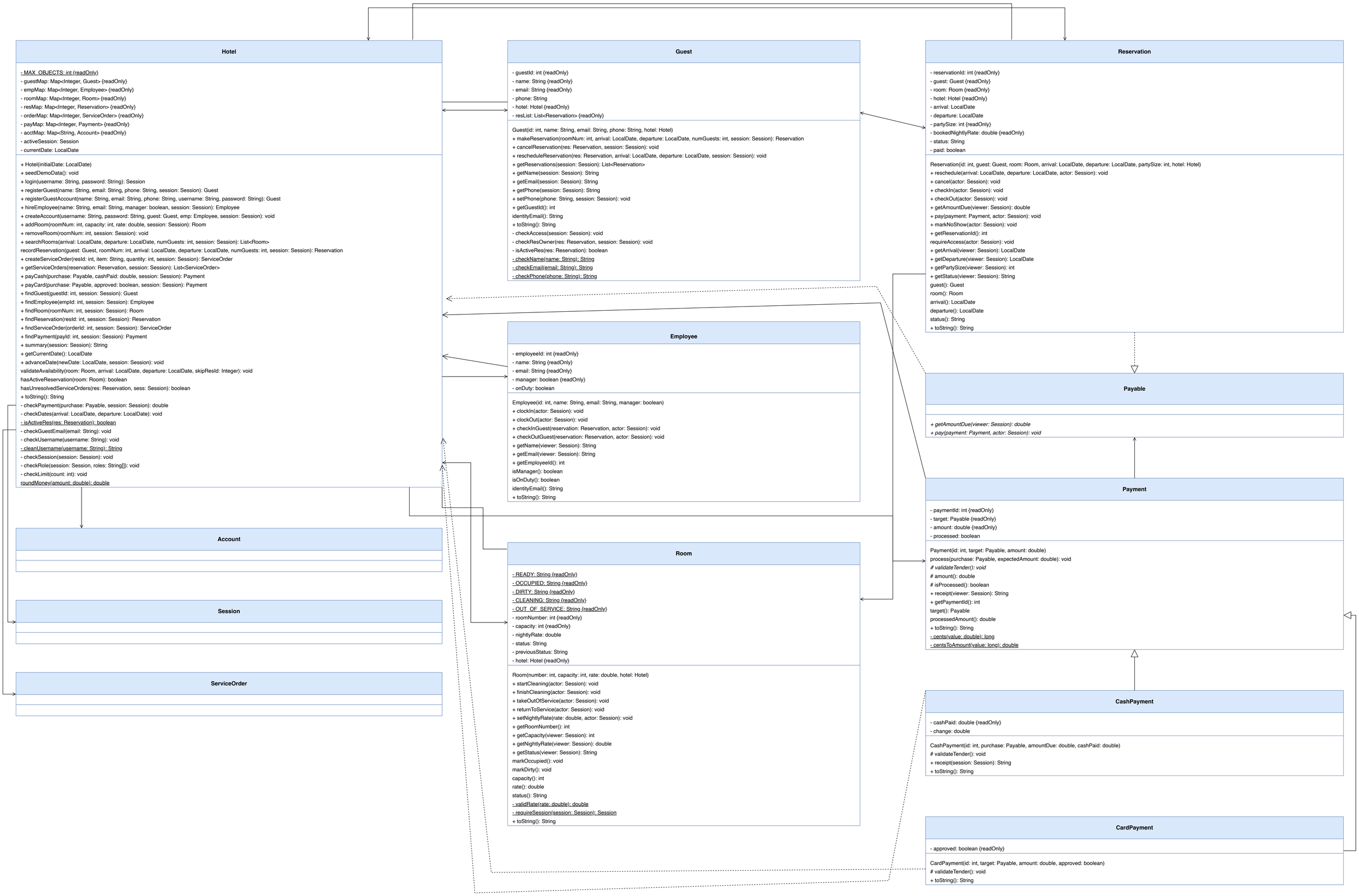
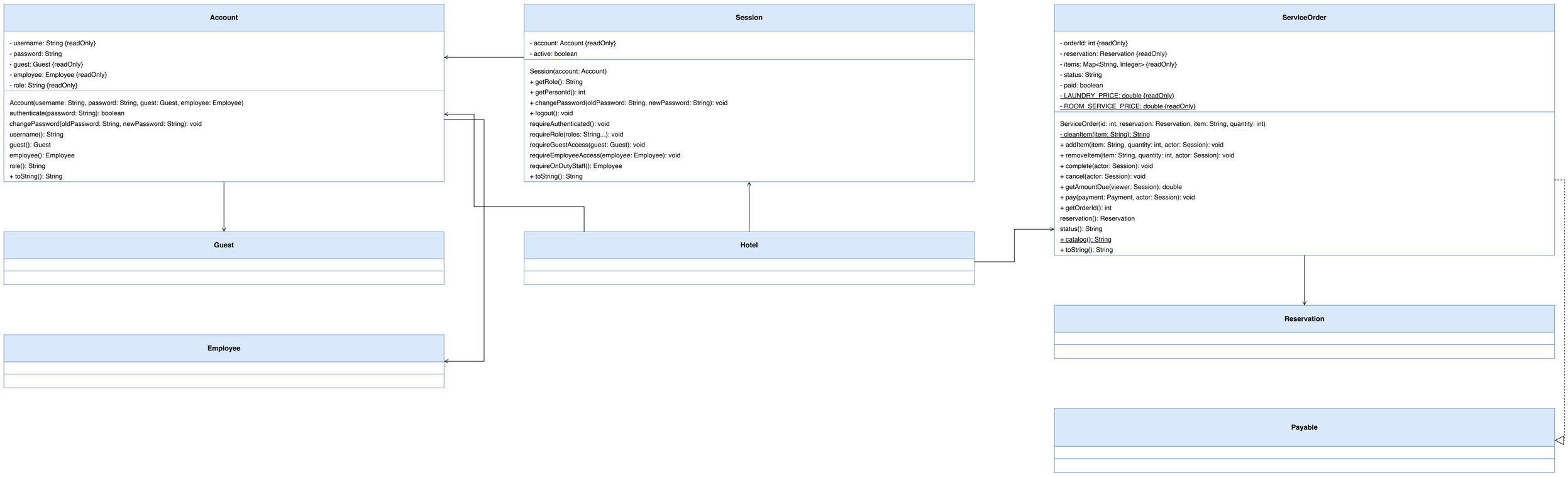
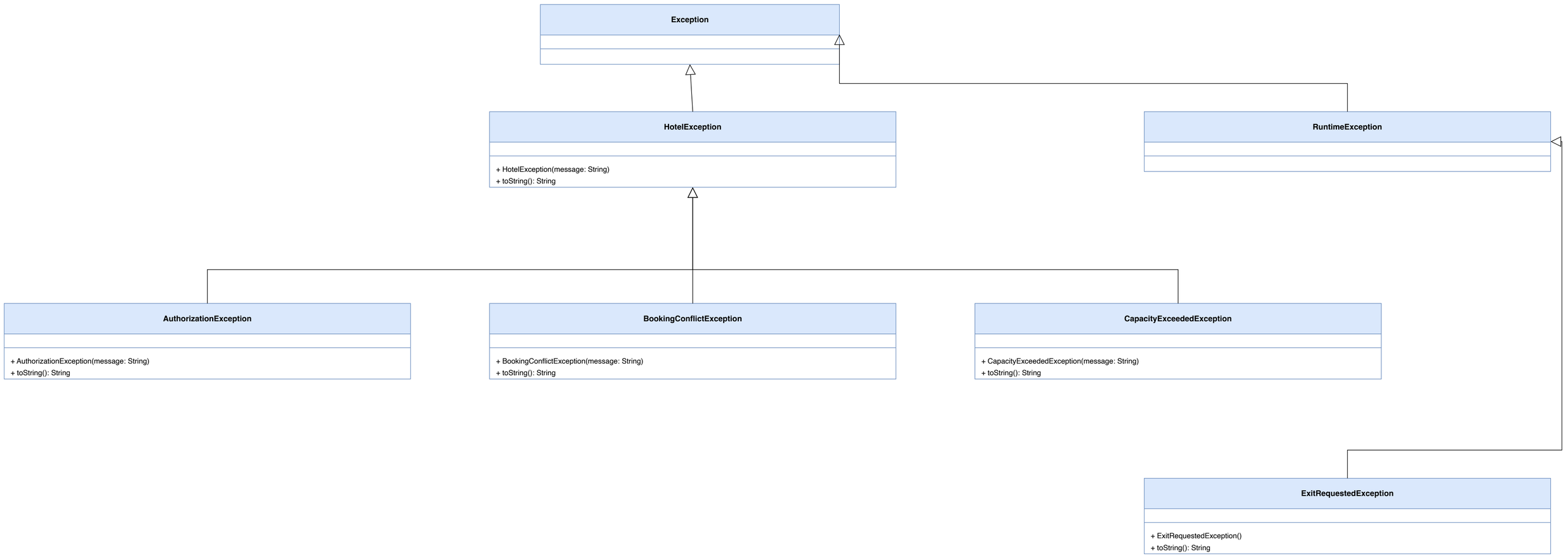
Model


Hotel
- MAX_OBJECTS: int {readOnly}
- guestMap: Map<Integer, Guest> {readOnly}
- empMap: Map<Integer, Employee> {readOnly}
- roomMap: Map<Integer, Room> {readOnly}
- resMap: Map<Integer, Reservation> {readOnly}
- orderMap: Map<Integer, ServiceOrder> {readOnly}
- payMap: Map<Integer, Payment> {readOnly}
- acctMap: Map<String, Account> {readOnly}
- activeSession: Session
- currentDate: LocalDate
+ Hotel(initialDate: LocalDate)
+ seedDemoData(): void
+ login(username: String, password: String): Session
+ registerGuest(name: String, email: String, phone: String, session: Session): Guest
+ registerGuestAccount(name: String, email: String, phone: String, username: String, password: String): Guest
+ hireEmployee(name: String, email: String, manager: boolean, session: Session): Employee
+ createAccount(username: String, password: String, guest: Guest, emp: Employee, session: Session): void
+ addRoom(roomNum: int, capacity: int, rate: double, session: Session): Room
+ removeRoom(roomNum: int, session: Session): void
+ searchRooms(arrival: LocalDate, departure: LocalDate, numGuests: int, session: Session): List<Room>
recordReservation(guest: Guest, roomNum: int, arrival: LocalDate, departure: LocalDate, numGuests: int, session: Session): Reservation
+ createServiceOrder(resId: int, item: String, quantity: int, session: Session): ServiceOrder
+ getServiceOrders(reservation: Reservation, session: Session): List<ServiceOrder>
+ payCash(purchase: Payable, cashPaid: double, session: Session): Payment
+ payCard(purchase: Payable, approved: boolean, session: Session): Payment
+ findGuest(guestId: int, session: Session): Guest
+ findEmployee(empId: int, session: Session): Employee
+ findRoom(roomNum: int, session: Session): Room
+ findReservation(resId: int, session: Session): Reservation
+ findServiceOrder(orderId: int, session: Session): ServiceOrder
+ findPayment(payId: int, session: Session): Payment
+ summary(session: Session): String
+ getCurrentDate(): LocalDate
+ advanceDate(newDate: LocalDate, session: Session): void
validateAvailability(room: Room, arrival: LocalDate, departure: LocalDate, skipResId: Integer): void
hasActiveReservation(room: Room): boolean
hasUnresolvedServiceOrders(res: Reservation, sess: Session): boolean
+ toString(): String
- checkPayment(purchase: Payable, session: Session): double
- checkDates(arrival: LocalDate, departure: LocalDate): void
- isActiveRes(res: Reservation): boolean
- checkGuestEmail(email: String): void
- checkUsername(username: String): void
- cleanUsername(username: String): String
- checkSession(session: Session): void
- checkRole(session: Session, roles: String[]): void
- checkLimit(count: int): void
roundMoney(amount: double): double
Guest
- guestId: int {readOnly}
- name: String {readOnly}
- email: String {readOnly}
- phone: String
- hotel: Hotel {readOnly}
- resList: List<Reservation> {readOnly}
Guest(id: int, name: String, email: String, phone: String, hotel: Hotel)
+ makeReservation(roomNum: int, arrival: LocalDate, departure: LocalDate, numGuests: int, session: Session): Reservation
+ cancelReservation(res: Reservation, session: Session): void
+ rescheduleReservation(res: Reservation, arrival: LocalDate, departure: LocalDate, session: Session): void
+ getReservations(session: Session): List<Reservation>
+ getName(session: Session): String
+ getEmail(session: Session): String
+ getPhone(session: Session): String
+ setPhone(phone: String, session: Session): void
+ getGuestId(): int
identityEmail(): String
+ toString(): String
- checkAccess(session: Session): void
- checkResOwner(res: Reservation, session: Session): void
- isActiveRes(res: Reservation): boolean
- checkName(name: String): String
- checkEmail(email: String): String
- checkPhone(phone: String): String
Reservation
- reservationId: int {readOnly}
- guest: Guest {readOnly}
- room: Room {readOnly}
- hotel: Hotel {readOnly}
- arrival: LocalDate
- departure: LocalDate
- partySize: int {readOnly}
- bookedNightlyRate: double {readOnly}
- status: String
- paid: boolean
Reservation(id: int, guest: Guest, room: Room, arrival: LocalDate, departure: LocalDate, partySize: int, hotel: Hotel)
+ reschedule(arrival: LocalDate, departure: LocalDate, actor: Session): void
+ cancel(actor: Session): void
+ checkIn(actor: Session): void
+ checkOut(actor: Session): void
+ getAmountDue(viewer: Session): double
+ pay(payment: Payment, actor: Session): void
+ markNoShow(actor: Session): void
+ getReservationId(): int
requireAccess(actor: Session): void
+ getArrival(viewer: Session): LocalDate
+ getDeparture(viewer: Session): LocalDate
+ getPartySize(viewer: Session): int
+ getStatus(viewer: Session): String
guest(): Guest
room(): Room
arrival(): LocalDate
departure(): LocalDate
status(): String
+ toString(): String
Employee
- employeeId: int {readOnly}
- name: String {readOnly}
- email: String {readOnly}
- manager: boolean {readOnly}
- onDuty: boolean
Employee(id: int, name: String, email: String, manager: boolean)
+ clockIn(actor: Session): void
+ clockOut(actor: Session): void
+ checkInGuest(reservation: Reservation, actor: Session): void
+ checkOutGuest(reservation: Reservation, actor: Session): void
+ getName(viewer: Session): String
+ getEmail(viewer: Session): String
+ getEmployeeId(): int
isManager(): boolean
isOnDuty(): boolean
identityEmail(): String
+ toString(): String
Payable
+ getAmountDue(viewer: Session): double
+ pay(payment: Payment, actor: Session): void
Payment
- paymentId: int {readOnly}
- target: Payable {readOnly}
- amount: double {readOnly}
- processed: boolean
Payment(id: int, target: Payable, amount: double)
process(purchase: Payable, expectedAmount: double): void
validateTender(): void
amount(): double
isProcessed(): boolean
+ receipt(viewer: Session): String
+ getPaymentId(): int
target(): Payable
processedAmount(): double
+ toString(): String
- cents(value: double): long
- centsToAmount(value: long): double
Account
Session
ServiceOrder
Room
- READY: String {readOnly}
- OCCUPIED: String {readOnly}
- DIRTY: String {readOnly}
- CLEANING: String {readOnly}
- OUT_OF_SERVICE: String {readOnly}
- roomNumber: int {readOnly}
- capacity: int {readOnly}
- nightlyRate: double
- status: String
- previousStatus: String
- hotel: Hotel {readOnly}
Room(number: int, capacity: int, rate: double, hotel: Hotel)
+ startCleaning(actor: Session): void
+ finishCleaning(actor: Session): void
+ takeOutOfService(actor: Session): void
+ returnToService(actor: Session): void
+ setNightlyRate(rate: double, actor: Session): void
+ getRoomNumber(): int
+ getCapacity(viewer: Session): int
+ getNightlyRate(viewer: Session): double
+ getStatus(viewer: Session): String
markOccupied(): void
markDirty(): void
capacity(): int
rate(): double
status(): String
- validRate(rate: double): double
- requireSession(session: Session): Session
+ toString(): String
CashPayment
- cashPaid: double {readOnly}
- change: double
CashPayment(id: int, purchase: Payable, amountDue: double, cashPaid: double)
validateTender(): void
+ receipt(session: Session): String
+ toString(): String
CardPayment
- approved: boolean {readOnly}
CardPayment(id: int, target: Payable, amount: double, approved: boolean)
validateTender(): void
+ toString(): String

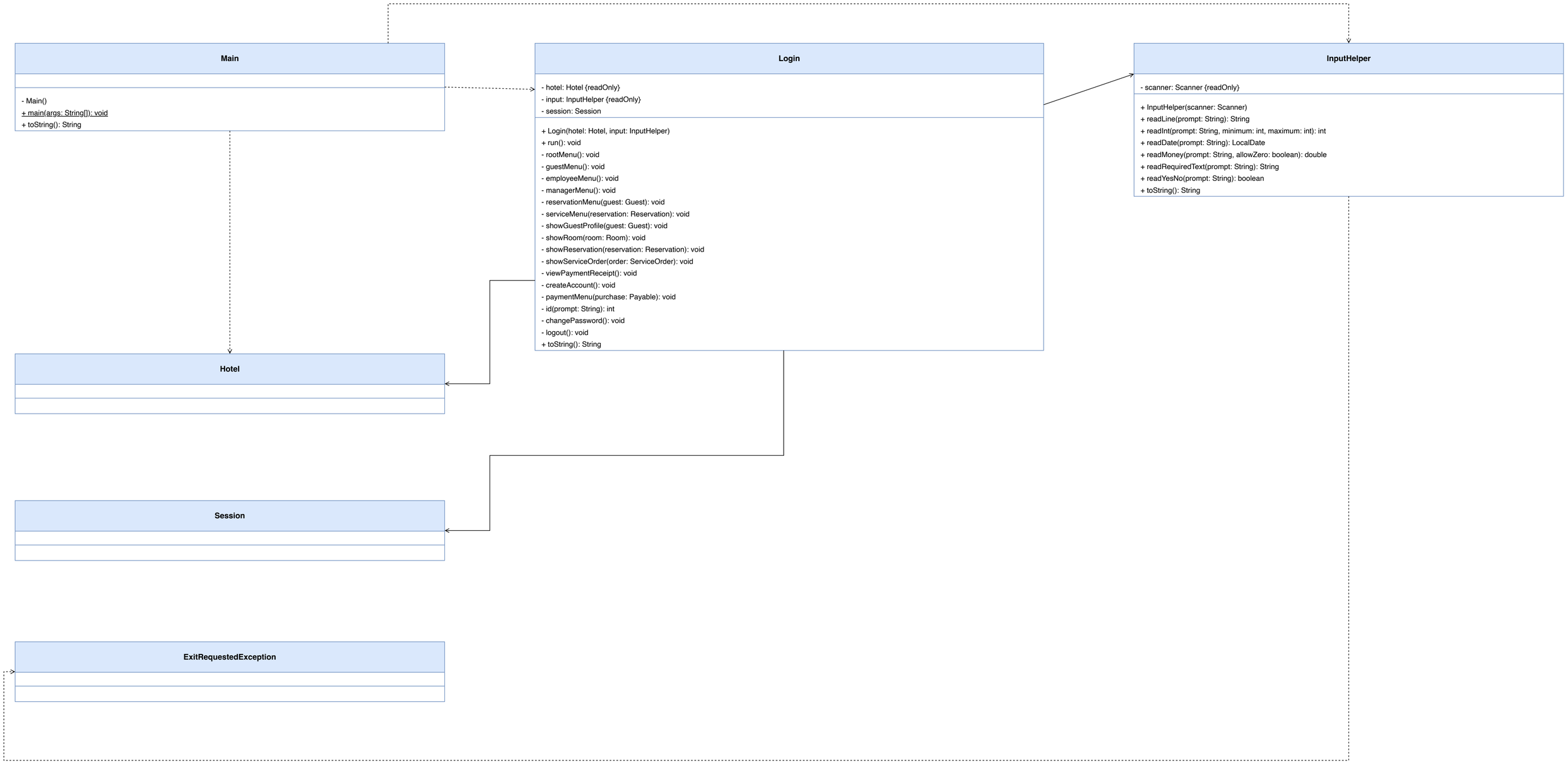
Accounts & Services



Exceptions


Exception
HotelException
+ HotelException(message: String)
+ toString(): String
RuntimeException
AuthorizationException
+ AuthorizationException(message: String)
+ toString(): String
BookingConflictException
+ BookingConflictException(message: String)
+ toString(): String
CapacityExceededException
+ CapacityExceededException(message: String)
+ toString(): String
ExitRequestedException
+ ExitRequestedException()
+ toString(): String

App & UI


Main
- Main()
+ main(args: String[]): void
+ toString(): String
Login
- hotel: Hotel {readOnly}
- input: InputHelper {readOnly}
- session: Session
+ Login(hotel: Hotel, input: InputHelper)
+ run(): void
- rootMenu(): void
- guestMenu(): void
- employeeMenu(): void
- managerMenu(): void
- reservationMenu(guest: Guest): void
- serviceMenu(reservation: Reservation): void
- showGuestProfile(guest: Guest): void
- showRoom(room: Room): void
- showReservation(reservation: Reservation): void
- showServiceOrder(order: ServiceOrder): void
- viewPaymentReceipt(): void
- createAccount(): void
- paymentMenu(purchase: Payable): void
- id(prompt: String): int
- changePassword(): void
- logout(): void
+ toString(): String
InputHelper
- scanner: Scanner {readOnly}
+ InputHelper(scanner: Scanner)
+ readLine(prompt: String): String
+ readInt(prompt: String, minimum: int, maximum: int): int
+ readDate(prompt: String): LocalDate
+ readMoney(prompt: String, allowZero: boolean): double
+ readRequiredText(prompt: String): String
+ readYesNo(prompt: String): boolean
+ toString(): String
Hotel
Session
ExitRequestedException

Tests

AuthorizationTest

- hotel: Hotel

setUp(): void {BeforeEach}
blocksOtherGuestAccessTest(): void {Test}
blocksStaffFromManagerActionsTest(): void {Test}
signupCreatesGuestAccountsTest(): void {Test}
blocksOldSessionsTest(): void {Test}
protectsGuestDataTest(): void {Test}

GuestReservationTest

- hotel: Hotel
- guest: Guest
- session: Session

setUp(): void {BeforeEach}
allowsBackToBackStaysTest(): void {Test}
rejectsBadBookingsTest(): void {Test}
keepsDatesAfterFailedChangeTest(): void {Test}
preventsChangesToPaidBookingsTest(): void {Test}
keepsBookingHistoryTest(): void {Test}

InputHelperTest

- helper(script: String): InputHelper
malformedInputIsRetried(): void {Test}
exitWorksAtEveryPromptType(): void {Test}
eofEndsInputNormally(): void {Test}
rawLinePreservesPasswordWhitespace(): void {Test}

HotelTest

- hotel: Hotel
- managerSession: Session

setUp(): void {BeforeEach}
roomRegistryStopsAtOneHundredTest(): void {Test}
duplicateRoomNumberIsRejectedTest(): void {Test}
duplicateGuestEmailIsRejectedTest(): void {Test}
failedGuestAccountRegistrationIsAtomicTest(): void {Test}

StaffRoomTest

- hotel: Hotel
- staffSession: Session
- managerSession: Session

setUp(): void {BeforeEach}
dutyChangesRequireOwnSession(): void {Test}
checkInRequiresDutyPaymentDateAndReadyRoom(): void {Test}
checkoutRequiresDepartureAndResolvedServices(): void {Test}
housekeepingRestoresRoomReadiness(): void {Test}
maintenanceRequiresManagerAndNoActiveBooking(): void {Test}
dateAdvancementDoesNotSkipConfirmedArrivals(): void {Test}

LoginTest

- hotel: Hotel

setUp(): void {BeforeEach}
passwordLoginByRole(): void {Test}
invalidLoginAndBusinessErrorsAllowRetry(): void {Test}
backAndLogoutReturnToExpectedMenus(): void {Test}
exitDuringFormCreatesNoPartialRecord(): void {Test}
guestProfileAndPhoneUpdateAreAvailable(): void {Test}
managerCanGiveNewEmployeeALoginAccount(): void {Test}
registeredGuestCanReceiveALoginAccount(): void {Test}
managerRoomActionsUpdateTheHotel(): void {Test}
reservationDetailsAndChangesAreAvailable(): void {Test}
serviceOrderEditingAndPaymentAreAvailable(): void {Test}
serviceOrdersStayWithinTheSelectedReservation(): void {Test}
receiptLookupPreservesPaymentOwnership(): void {Test}
- createCheckedInReservations(roomNumbers: int...): Reservation[]
- run(script: String): String

PaymentServiceTest

- hotel: Hotel
- guest: Guest
- employee: Employee
- guestSession: Session
- employeeSession: Session
- reservation: Reservation

setUp(): void {BeforeEach}
serviceOrderItemsUpdateBalanceTest(): void {Test}
invalidServiceChangesAreRejectedTest(): void {Test}
completedServiceOrderCanBePaidTest(): void {Test}
declinedCardLeavesOrderUnpaidTest(): void {Test}
cashPaymentReturnsCorrectChangeTest(): void {Test}
openServiceOrderCannotBePaidTest(): void {Test}